REFERENCE · CHAPTER 5 DEEP DIVE

Row & Column Access Controls: Stateless HTTP (BigQuery) vs. Stateful Pooled TCP (Cloud SQL)

Why BigQuery's stateless per-request HTTP connection model and Cloud SQL's stateful pooled TCP sockets fundamentally change how an AI agent enforces Row-Level and Column-Level Security.

The difference between enforcing row and column isolation in an analytical data warehouse versus an operational database comes down to **BigQuery's stateless, per-request HTTP API connection model** versus **Cloud SQL's stateful, long-lived TCP connection-pooling model**.

Because BigQuery evaluates an IAM/OAuth token on every individual HTTP query job, it natively enforces IAM-based Row-Level Security (RLS) and Column-Level Security (Policy Tags) per request. In contrast, Cloud SQL relies on a persistent pool of TCP connections authenticated under a single shared database role, meaning the database engine cannot natively distinguish which end user is behind an agent's query without re-authenticating or breaking the connection pool.

1. How BigQuery Connects: Stateless Per-Request Identity

When an ADK agent calls BigQuery (either via the BigQuery Remote MCP Server or MCP Toolbox with `useClientOAuth: true`), there is no logged-in session held open on a socket:

- Each MCP tool call is an independent, stateless HTTP/gRPC request carrying `Authorization: Bearer <End-User-OAuth-Token>` in the header.
- Even if 1,000 users talk to one agent, the agent or MCP server attaches each user's OAuth token to their HTTP request, and BigQuery inspects the IAM principal inside that token for that single query job.
- BigQuery automatically evaluates its **Row Access Policies** (filtering rows matching the user's IAM identity via `SESSION_USER()` or app-level attributes via `SECURE_CONTEXT()`) and **Column-Level Policy Tags** (masking or denying sensitive PII columns) before scanning the table.

2. How Cloud SQL Connects: Stateful Pooled Sockets

Relational databases such as PostgreSQL, MySQL, and SQL Server were designed around stateful TCP connections where authentication happens once during the initial connection handshake:

- Opening a new TCP connection and running an IAM handshake per query would exhaust `max_connections` and add high latency. Creating a separate database user/role for every end user does not scale in web or agent applications.
- Instead, MCP Toolbox (or your application backend) opens a warm pool of persistent connections using one minimally-scoped service account or database role for all requests—so to the database, every query looks like it came from the exact same identity.
- Because that single database role must have `SELECT` privileges on the multi-tenant table to serve all users, native database `GRANT` permissions alone cannot stop User A's agent session from querying User B's rows over that shared connection.

Architectural Comparison: BigQuery vs. Cloud SQL

ARCHITECTURAL DIMENSION	BIGQUERY (DATA WAREHOUSE)	CLOUD SQL — POSTGRESQL / MYSQL / SQL SERVER (OLTP)
How the Connection is Made	Stateless HTTP/gRPC API calls. There is no persistent socket or connection pool; every query job is self-contained.	Stateful, long-lived TCP sockets managed via a connection pool (inside MCP Toolbox or Cloud SQL Auth Proxy).
Identity Seen by the Database	The individual End User's IAM Principal (`user:alice@company.com` or `group:...`) passed via OAuth header on each request (`useClientOAuth: true`).	A single shared Database Role / Service Account that owns the connection pool serving all application users.
Row-Level Security (RLS) Mechanism	IAM-Native DDL Policies (`CREATE ROW ACCESS POLICY ... GRANT TO ('user:...')`) evaluated automatically per API call. Also supports `SECURE_CONTEXT()` for app-level attributes.	DB-Internal Roles (`CREATE POLICY` + `FORCE ROW LEVEL SECURITY` with transaction-local `set_config` in PG) or Parameterized Secure Views (`@local_customer_id`) because end users are rarely individual DB/IAM users.
Column-Level Security (CLS) Mechanism	Dataplex / Data Catalog Policy Tags bound directly to IAM principals; unauthorized columns are blocked or masked per query.	Table/Column `GRANT SELECT (col1, col2)` tied to the database role on the connection—meaning all users sharing the pool get the exact same column visibility.

ARCHITECTURAL DIMENSION	BIGQUERY (DATA WAREHOUSE)	CLOUD SQL — POSTGRESQL / MYSQL / SQL SERVER (OLTP)
Why Raw `execute_sql` Differs in Risk	If End-User OAuth passthrough is enabled, `execute_sql` is still bounded by that specific user's IAM Row and Column policies in BigQuery.	Because the shared connection pool has permission to read the entire table across all tenants, raw `execute_sql` allows a prompt-injected agent to read any customer's rows.

CLOUD SQL ISOLATION PATTERNS

How to Enforce Row and Column Isolation in Cloud SQL for Agents

Three complementary mechanisms to isolate rows and columns over a single shared database role and connection pool.

Since you cannot rely on per-connection IAM identities in a Cloud SQL connection pool, Google Cloud provides two high-level solution patterns—which you can reinforce with transaction-local PostgreSQL Row-Level Security—to enforce row and column isolation while keeping a single shared database role and connection pool:

SOLUTION PATTERN	HOW IT WORKS OVER A SHARED CONNECTION POOL	WHEN TO USE IT
1. MCP Toolbox Parameterized Tools + `bound_params` / `authServices`	You define fixed SQL statements in `tools.yaml` (specifying only the allowed columns in `SELECT col1, col2` and locking the `WHERE user_id = $1` filter). Your ADK agent or MCP Toolbox `authServices` binds `user_id` server-side from the authenticated session so the LLM cannot alter columns or rows.	Best for custom transactional and lookup workflows where the agent maps intents to known query patterns.
2. Cloud SQL Parameterized Secure Views	You create a restricted view whose row filter is a named parameter and grant the shared connection pool role access only to the view, not the base table. When using the Conversational Analytics `QueryData` API or MCP Toolbox, the application injects the verified user parameter so even AI-generated ad-hoc SQL is strictly confined to that user's rows and columns. The syntax differs by engine: Cloud SQL for MySQL (8.0.43 and later) uses session variables (`CREATE VIEW v_orders AS SELECT … FROM orders WHERE customer_id = @local_customer_id`); Cloud SQL for PostgreSQL requires the `cloudsql.enable_parameterized_views` flag and the `parameterized_views` extension, and uses `WITH (security_barrier)` views with `$@` -prefixed parameters (`WHERE customer_id = $@app_user_id`).	Best when you need open-ended natural-language-to-SQL analytics over Cloud SQL (supported in Cloud SQL for MySQL 8.0.43+ and PostgreSQL) without exposing other tenants' data.

SOLUTION PATTERN	HOW IT WORKS OVER A SHARED CONNECTION POOL	WHEN TO USE IT
3. Transaction-Local PostgreSQL `set_config` + `FORCE ROW LEVEL SECURITY`	Inside each transaction on the shared pool (`BEGIN ... COMMIT`), the tool server stamps the verified user ID via `SELECT set_config('app.current_user_id', $1, true)` (`is_local = true` / `SET LOCAL`). A `RESTRICTIVE` policy with `FORCE ROW LEVEL SECURITY` filters every query against `current_setting('app.current_user_id', true)` , and the setting automatically resets when the transaction ends before the socket returns to the pool.	Best as an engine-level defense-in-depth safety net in Cloud SQL / AlloyDB for PostgreSQL so even a newly added tool that forgets a `WHERE` clause cannot leak rows across tenants.

VISUAL ARCHITECTURE MAP

Side-by-Side Architecture: BigQuery (SESSION_USER( )) vs. Cloud SQL / AlloyDB (SET LOCAL + FORCE RLS)

How identity propagation, Row-Level Security, and Column-Level Security execute step by step in both engines.

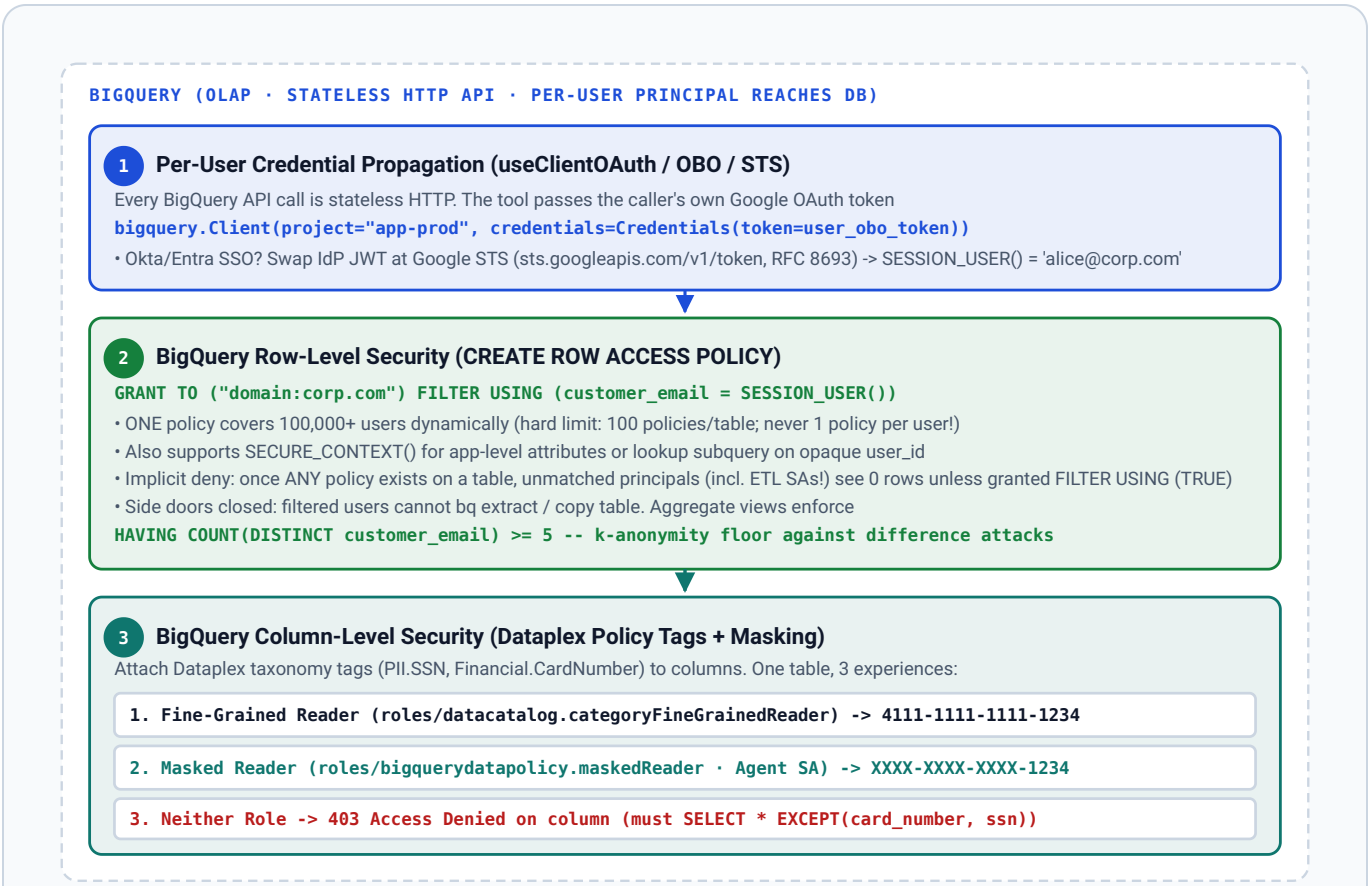


Figure 1A — Row-Level Security (RLS) and Column-Level Security (CLS) Architecture, BigQuery: SESSION_USER() + Dataplex Policy Tags.

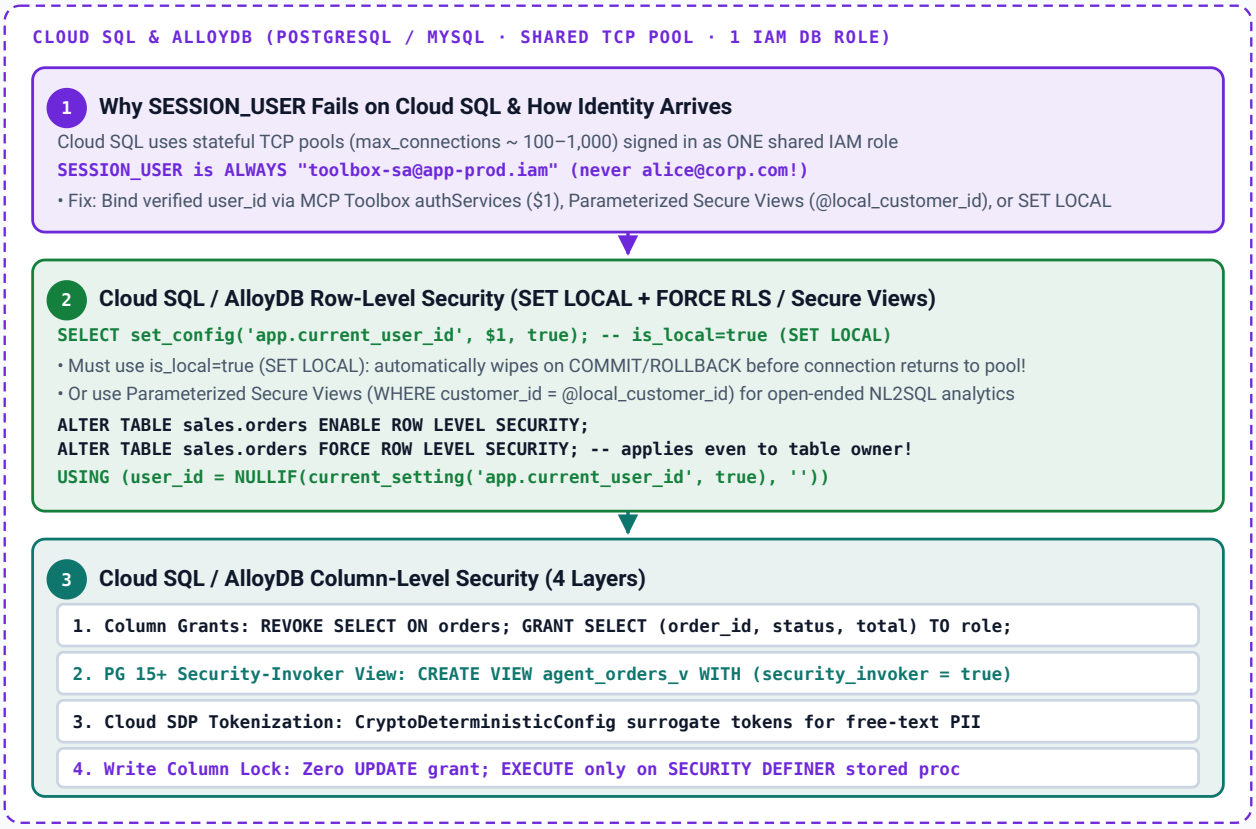


Figure 1B — Row-Level Security (RLS) and Column-Level Security (CLS) Architecture, Cloud SQL / AlloyDB: set_config(..., true) SET LOCAL + FORCE RLS + Parameterized Secure Views + Column Grants.

Critical Implementation Gotchas: BigQuery vs. Cloud SQL / AlloyDB

GOVERNANCE DIMENSION	BIGQUERY (OLAP / STATELESS HTTP)	CLOUD SQL & ALLOYDB FOR POSTGRESQL (OLTP / POOLED TCP)
1. What `SESSION_USER` Returns	Returns the authenticated human's email (`alice@corp.com`) or Workforce principal when called with OBO / `useClientOAuth: true` credentials; returns the service account only for background ETL jobs.	**Always returns the shared pool service account** (`toolbox-sa@app-prod.iam`). A policy checking `SESSION_USER` cannot distinguish Alice from Bob.
2. Critical RLS Gotchas & Footguns	• **Hard limit of 100 policies/table:** Write 1 policy with `SESSION_USER()` , never 1 policy per user. • **Implicit Zero-Row Deny:** Once *any* policy exists on a table, unmatched principals (including nightly ETL SAs) see `0` rows unless granted an explicit `FILTER USING (TRUE)` policy. • **Without RLS, OBO does not filter rows:** `roles/bigquery.dataViewer` is table-wide;	• **Never use plain `SET` on a pool:** Plain `SET app.current_user_id` persists on the TCP socket after the query finishes, leaking Alice's rows to Bob's next request. Always pass `is_local = true` (`SET LOCAL`) inside a transaction. • **Guard empty strings with `NULLIF` :** Use `NULLIF(current_setting('app.tenant_id',`

GOVERNANCE DIMENSION	BIGQUERY (OLAP / STATELESS HTTP)	CLOUD SQL & ALLOYDB FOR POSTGRESQL (OLTP / POOLED TCP)
	without a Row Access Policy, `SELECT *` returns all tenants' rows even with `user_creds`.	`true), ''):uuid` so an unset GUC returns `0` rows instead of throwing an invalid UUID cast error.
3. Dynamic Data Masking & PII Tokenization	**BigQuery Data Policies (`roles/bigquerydatapolicy.maskedReader`):** Attach a masking rule (`SHA256`, `DEFAULT_MASKING_VALUE`, `LAST_FOUR_CHARACTERS`, `NULLIFY`) to the Dataplex Policy Tag. The AI agent's query succeeds without failing on `SELECT *`, receiving masked values automatically.	• **In SQL View:** Mask structured columns inline (`'***-**-' || right(ssn, 4) AS ssn_masked`) on a `WITH (security_invoker = true)` view (never omit `security_invoker = true`, or the view runs as the view owner and bypasses RLS). • **In MCP / ADK Callback:** Call **Cloud Sensitive Data Protection (SDP)** `deidentify_content` or deterministic AES-SIV surrogate tokenization (`CryptoDeterministicConfig`).
4. Aggregate & Side-Door Leak Prevention	Filtered users are blocked from `bq extract`, table copy, and unfiltered Storage Read API. On aggregate views exposed to agents, enforce a **`k`-anonymity threshold (`HAVING COUNT(DISTINCT customer_email) >= 5`)** so difference attacks cannot isolate 1 employee's salary.	Enforce `LIMIT 50` on every read tool, point read tools at a read replica with `default_transaction_read_only = on`, `statement_timeout = '5s'`, and `CONNECTION LIMIT 20` on the agent's database role.
5. Write / Mutation Column Control	Agents should not mutate OLAP tables directly; if required, append proposals to an insert-only staging table merged by a scheduled batch job.	Revoke all `INSERT, UPDATE, DELETE` on tables from the agent role. Grant `EXECUTE` only on a **`SECURITY DEFINER` stored procedure** (with `SET search_path = sales, pg_temp`) that checks row ownership, idempotency `request_id`, and autonomous dollar caps (`<= $100`).

EXECUTION MODES & REFERENCE CODE

How Verified Identity Reaches RLS Across All 3 Agent Execution Modes

Designing for interactive browser sessions (Mode 1), event-driven webhooks (Mode 2), and scheduled autonomous jobs (Mode 3), followed by runnable SQL and ADK 2.x reference implementations.

Designing RLS only for an interactive browser user (`Mode 1`) leaves a gap when an autonomous agent wakes up at 3:00 AM from a webhook, email, or schedule (`Modes 2 & 3`) with no live browser JWT. Across all three modes, the rule is the same: **resolve and verify the identity envelope before the LLM runs, lock it into out-of-band state, and remove `user_id` from the LLM's tool schema.**

MODE 01 · INTERACTIVE CHAT AGENT

Human Online at Browser Now

1. Browser sends `Authorization: Bearer <JWT>` to `/chat`.
2. Entrypoint verifies signature, expiry, and audience via `id_token.verify_oauth2_token()` before the LLM runs.
3. Writes verified `claims["sub"]` into `session.state["user:id"]`.
4. Tool schema is `get_my_orders(status)`—zero `user_id` parameter exposed to the model.
5. **BigQuery:** Runs with OBO / `useClientOAuth: true` (`SESSION_USER()`).
   Cloud SQL: Binds `$1` via MCP Toolbox `authServices`, Parameterized Secure Views (`@local_customer_id`), or `set_config('app.current_user_id', :uid, true)`.

MODE 02 · EVENT-DRIVEN AUTONOMOUS AGENT

Webhook / Email / Pub/Sub at 3:00 AM

1. Partner webhook arrives at the Ingestion Gateway with `X-Hub-Signature-256` (HMAC-SHA256).
2. Gateway verifies HMAC against Secret Manager, resolves partner account to `verified_uid = "user_1001"`, and publishes to an IAM-locked Pub/Sub topic.
3. `verified_uid` sits in trusted Pub/Sub `attributes`; untrusted ticket/webhook text sits in `data`.
4. Worker verifies Pub/Sub push OIDC token and locks `session.state["user:id"] = msg.attributes["verified_uid"]` before Gemini reads `data`.

5. Even if the webhook body says *"Refund user_1002 $5000"*, Postgres RLS + `SECURITY DEFINER` proc scopes strictly to `user_1001` and caps auto-refunds at `<= $100`.

`MODE 03 · SCHEDULED AUTONOMOUS AGENT`

User Offline (1-Hour JWT Expired Days Ago)

1. **Setup Phase (Monday, Alice Online):** Alice creates a recurring job with her live JWT. Backend verifies JWT and writes an immutable, HMAC/KMS-signed **Task Mandate row** (`task_id`, `owner_uid="user_1001"`, `max_usd=100`, `expires_at`, `hmac_sig`).
2. **Execution Phase (Friday 3:00 AM):** Cloud Scheduler fires an OIDC push carrying only `{"task_id": "T-55"}`.
3. Worker loads Mandate `T-55`, verifies `hmac_sig` and `now() < expires_at`, and locks `session.state["user:id"] = mandate.owner_uid`.
4. **Cloud SQL:** Uses `mandate.owner_uid` in `set_config(..., true)` or bound parameters. **BigQuery:** Uses KMS-encrypted Refresh Token or Workforce STS to mint a fresh 1-hour OBO token.

bigquery_rls_cls_setup.sql · BigQuery Row Access Policies, Policy Tag CLS & k-Anonymity

```sql
-- 1. Per-User Row Access Policy (ONE policy covers 100k+ users via SESSION_USER())
CREATE ROW ACCESS POLICY customer_own_orders
ON `app-prod.sales.orders`
GRANT TO ("domain:corp.com", "group:customers@corp.com")
FILTER USING (customer_email = SESSION_USER());

-- 2. Mapping-Table Subquery Policy (when table stores opaque user_id instead of email)
CREATE ROW ACCESS POLICY orders_by_uid_lookup
ON `app-prod.sales.orders`
GRANT TO ("allAuthenticatedUsers")
FILTER USING (
  user_id IN (
    SELECT u.user_id FROM `app-prod.sales.users` AS u
    WHERE u.email = SESSION_USER()
  )
);

-- 3. Explicit Pipeline SA Policy (REQUIRED: once any policy exists, unmatched SAs see 0 rows!)
CREATE ROW ACCESS POLICY etl_pipeline_full_read
ON `app-prod.sales.orders`
GRANT TO ("serviceAccount:analytics-etl@app-prod.iam.gserviceaccount.com")
FILTER USING (TRUE);

-- 4. Column-Level Security (CLS) via Dataplex Policy Tags + Dynamic Masking:
--    Attach policyTag "projects/app-prod/locations/us/taxonomies/1/policyTags/card_num"
--    Grant Agent SA: roles/bigquerydatapolicy.maskedReader (sees XXXX-XXXX-XXXX-1234, never raw PAN!)
SELECT order_id, customer_email, card_number, total
FROM `app-prod.sales.orders`
LIMIT 20;

-- 5. Aggregate View with k-Anonymity Floor (k >= 5 distinct users blocks difference attacks)
CREATE OR REPLACE VIEW `app-prod.agent_views.regional_order_stats_v` AS
```

```sql
SELECT
  region,
  status,
  COUNT(*) AS order_count,
  ROUND(AVG(total), 2) AS avg_order_total
FROM `app-prod.sales.orders`
GROUP BY region, status
HAVING COUNT(DISTINCT customer_email) >= 5;
```

bigquery_obo_sts_agent.py · Verifying JWT + RFC 8693 STS Exchange for BigQuery RLS/CLS

```python
import requests
from google.auth.transport import requests as google_requests
from google.cloud import bigquery
from google.oauth2 import id_token
from google.oauth2.credentials import Credentials

SERVICE_URL = "https://orders-agent-uc.a.run.app"
STS_URL = "https://sts.googleapis.com/v1/token"
WORKFORCE_AUD = "//iam.googleapis.com/locations/global/workforcePools/corp-pool/providers/entra-oidc"

def verify_caller_jwt(auth_header: str) -> dict:
    """Verifies signature, expiry & audience; extracts immutable 'sub' and 'email'."""
    raw_jwt = auth_header.removeprefix("Bearer ").strip()
    return id_token.verify_oauth2_token(raw_jwt, google_requests.Request(), audience=SERVICE_URL)

def exchange_idp_jwt_for_bq_creds(incoming_idp_jwt: str) -> Credentials:
    """RFC 8693 OBO Token Exchange via Google STS so SESSION_USER() = human caller."""
    resp = requests.post(
        STS_URL,
        json={
            "grantType": "urn:ietf:params:oauth:grant-type:token-exchange",
            "audience": WORKFORCE_AUD,
            "scope": "https://www.googleapis.com/auth/bigquery.readonly",
            "requestedTokenType": "urn:ietf:params:oauth:token-type:access_token",
            "subjectToken": incoming_idp_jwt,
            "subjectTokenType": "urn:ietf:params:oauth:token-type:jwt",
        },
        timeout=(2, 5),
    )
    resp.raise_for_status()
    return Credentials(token=resp.json()["access_token"])

def query_orders_with_bq_rls_and_cls(auth_header: str) -> list[dict]:
    claims = verify_caller_jwt(auth_header)
    raw_jwt = auth_header.removeprefix("Bearer ").strip()
    user_creds = exchange_idp_jwt_for_bq_creds(raw_jwt)

    # Query runs as Alice: BigQuery enforces Row Access Policy + Policy Tag CLS masking!
    user_bq = bigquery.Client(project="app-prod", credentials=user_creds)
    job_cfg = bigquery.QueryJobConfig(
        maximum_bytes_billed=100 * 1024 * 1024,  # 100 MB FinOps / LLM10 guardrail
        query_parameters=[bigquery.ScalarQueryParameter("email", "STRING", claims["email"])],
        labels={"acting_agent": "orders-agent", "caller": claims["sub"][:40]},
    )
    sql = """
      SELECT order_id, status, total, card_number
      FROM `app-prod.sales.orders`
      WHERE customer_email = @email
      ORDER BY created_at DESC LIMIT 50
```

```
    """
    return [dict(r) for r in user_bq.query(sql, job_config=job_cfg).result(timeout=20)]
```

cloudsql_rls_cls_setup.sql · Cloud SQL Column Grants, Secure Views, FORCE RLS & Stored Proc

```
-- 1. Column-Level Security (Layer 1): Revoke broad SELECT; grant ONLY safe columns
REVOKE ALL ON TABLE sales.orders FROM PUBLIC, "toolbox-sa@app-prod.iam";
GRANT SELECT (order_id, user_id, owner_email, status, total, updated_at)
  ON TABLE sales.orders TO "toolbox-sa@app-prod.iam";

-- 2a. Security-Invoker View with inline masking (PG 15+)
CREATE OR REPLACE VIEW sales.agent_orders_v WITH (security_invoker = true) AS
SELECT
  order_id,
  user_id,
  owner_email,
  status,
  total,
  '***-***-' || right(contact_phone, 4) AS contact_phone_masked,
  updated_at
FROM sales.orders;
GRANT SELECT ON sales.agent_orders_v TO "toolbox-sa@app-prod.iam";

-- 2b. Cloud SQL Parameterized Secure View (for Conversational Analytics / NL2SQL)
-- Requires flag: cloudsql.enable_parameterized_views = on
-- Syntax note: PostgreSQL parameterized views use WITH (security_barrier) and $@param (plus the parameterized_
-- the @param form shown below is the Cloud SQL for MySQL 8.0.43+ syntax.
CREATE VIEW sales.v_customer_orders WITH (security_invoker = true) AS
SELECT order_id, status, total, updated_at
FROM sales.orders
WHERE user_id = @local_customer_id;

-- 3. Row-Level Security (RLS): ENABLE + FORCE ROW LEVEL SECURITY on shared pool role
ALTER TABLE sales.orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE sales.orders FORCE ROW LEVEL SECURITY;

CREATE POLICY agent_user_isolation ON sales.orders
  AS RESTRICTIVE
  FOR SELECT TO "toolbox-sa@app-prod.iam"
  USING (
    status <> 'DELETED'
    AND user_id = NULLIF(current_setting('app.current_user_id', true), '')
  );

-- 4. Write Column & Row Lock: Zero UPDATE/DELETE grants; mutate ONLY via SECURITY DEFINER proc
CREATE OR REPLACE FUNCTION sales.request_or_issue_refund(
  p_order_id TEXT,
  p_amount   NUMERIC,
  p_idem_key TEXT
) RETURNS JSONB
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = sales, pg_temp
AS $$
DECLARE
  v_uid   TEXT := NULLIF(current_setting('app.current_user_id', true), '');
  v_total NUMERIC;
BEGIN
  IF v_uid IS NULL THEN
    RAISE EXCEPTION 'Missing transaction user context (app.current_user_id)';
  END IF;
```

```
  IF EXISTS (SELECT 1 FROM sales.refund_ledger WHERE idempotency_key = p_idem_key) THEN
    RETURN jsonb_build_object('status', 'already_processed', 'idempotency_key', p_idem_key);
  END IF;
  SELECT total INTO v_total FROM sales.orders WHERE order_id = p_order_id AND user_id = v_uid FOR UPDATE;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'Order % not found or not owned by %', p_order_id, v_uid;
  END IF;
  IF p_amount > 100.00 THEN
    INSERT INTO sales.refund_approval_queue (idempotency_key, order_id, user_id, amount, status)
    VALUES (p_idem_key, p_order_id, v_uid, p_amount, 'PENDING_HUMAN_APPROVAL');
    RETURN jsonb_build_object('status', 'queued_for_human_approval', 'order_id', p_order_id);
  END IF;
  INSERT INTO sales.refund_ledger (idempotency_key, order_id, user_id, amount)
  VALUES (p_idem_key, p_order_id, v_uid, p_amount);
  UPDATE sales.orders SET status = 'REFUNDED', updated_at = now()
  WHERE order_id = p_order_id AND user_id = v_uid;
  RETURN jsonb_build_object('status', 'refunded', 'order_id', p_order_id, 'amount', p_amount);
END;
$$;
REVOKE ALL ON FUNCTION sales.request_or_issue_refund(TEXT, NUMERIC, TEXT) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION sales.request_or_issue_refund(TEXT, NUMERIC, TEXT) TO "toolbox-writer-sa@app-prod.iam
```

cloudsql_adk_rls_worker.py · Out-of-Band ToolContext + Transaction-Local SET LOCAL

```python
from typing import Literal
from google.adk import Agent
from google.adk.tools import ToolContext
from google.genai import types
from sqlalchemy import create_engine, text

# Shared Cloud SQL / AlloyDB pool signed in as least-privilege IAM role
db_engine = create_engine(
    "postgresql+pg8000://toolbox-sa%40app-prod.iam@10.10.2.50:5432/prod_db",
    pool_size=10,
)

# Notice: NO user_id parameter in the function signature! The LLM cannot see or spoof it.
def get_my_orders(
    status: Literal["OPEN", "SHIPPED", "REFUNDED"],
    tool_context: ToolContext,
) -> dict:
    """Fetch the authenticated caller's orders filtered by status."""
    # 1. Read verified user_id locked into ADK session state by the HTTP/PubSub entrypoint:
    verified_uid = tool_context.state.get("user:id")
    if not verified_uid:
        raise PermissionError("Missing verified user:id in session state")

    # 2. Borrow a pooled connection inside a transaction (BEGIN ... COMMIT):
    with db_engine.begin() as conn:
        # 3. Stamp verified_uid onto THIS transaction only (is_local=true -> SET LOCAL):
        conn.execute(
            text("SELECT set_config('app.current_user_id', :uid, true)"),
            {"uid": verified_uid},
        )
        # 4. Query the security_invoker view; Postgres FORCE RLS also enforces user_id match!
        rows = conn.execute(
            text("""
                SELECT order_id, status, total, contact_phone_masked, updated_at
                FROM sales.agent_orders_v
                WHERE status = :st
```

```
                ORDER BY updated_at DESC LIMIT 50
            """),
            {"st": status},
        ).mappings().all()
    # <-- Transaction ends here; app.current_user_id is wiped before returning to the pool!
    return {"orders": [dict(r) for r in rows]}

orders_agent = Agent(
    name="orders_agent",
    model="gemini-3.8-flash",
    mode="single_turn",
    generate_content_config=types.GenerateContentConfig(
        thinking_config=types.ThinkingConfig(thinking_level="LOW")
    ),
    tools=[get_my_orders],
)
```